CS 521: Fall 2026
Homework 3

Atharva Bhatt (bhatt13@illinois.edu)

Problem 1. (Adversarial Training)

The code is in a Colab notebook **here** and also on **GitHub**.

(a) Standard and ℓ_∞ adversarially trained models

Setup. Both models are a new PreActResNet-18 from `make_model()`, trained from scratch for 5 epochs on a subset of 10,000 CIFAR-10 training images (SGD, learning rate 0.01, momentum 0.9, weight decay $5 \cdot 10^{-4}$, cosine schedule, batch size 128). The standard model uses the cross-entropy loss on clean images. The ℓ_∞ adversarially trained model ("ℓ_∞-AT") uses the cross-entropy loss on PGD examples with $\epsilon = 8/255$ (10 PGD steps):

```
def single_norm_loss(model, x, y, norm):
    model.eval()
    adv_x = attack(model, x, y, norm)
    model.train()
    return F.cross_entropy(model(adv_x), y)
```

As the question says, the starter notebook of Problem 3 is the code base, so the training subset (10,000 images) and the evaluation set (the first 1,000 CIFAR-10 test images) are the ones given in that notebook. Accuracies (in %) are measured on the 1,000 test images, using PGD with 50 steps and 2 restarts unless stated otherwise (FGSM in part (b)).

Default budgets (ℓ_∞: 8/255, ℓ_2: 0.5, ℓ_1: 12):

Model	Clean	ℓ_∞ PGD	ℓ_2 PGD	ℓ_1 PGD	Union
Standard	57.8	0.2	8.8	1.6	0.2
ℓ_∞-AT	36.7	23.1	28.6	20.5	18.8

PGD-50 at different ϵ. Clean accuracy is 57.8 (standard) and 36.7 (ℓ_∞-AT).

Norm	Model	ϵ				
		1/255	2/255	4/255	8/255	16/255
ℓ_∞	Standard	37.5	21.9	4.8	0.2	0.0
ℓ_∞	ℓ_∞-AT	34.6	32.8	29.1	23.1	13.2
		0.25	0.5	1.0	2.0	
ℓ_2	Standard	26.4	8.8	0.5	0.0	
ℓ_2	ℓ_∞-AT	32.9	28.6	22.5	12.1	
		6	12	24		
ℓ_1	Standard	11.3	1.5	0.2		
ℓ_1	ℓ_∞-AT	26.6	20.4	11.9		

Observations.

- The standard model has the best clean accuracy (57.8%) but no robustness: 0.2% under ℓ_∞ PGD at 8/255, and already 4.8% at 4/255.

- Adversarial training gives 23.1% robust accuracy at 8/255, but the clean accuracy drops from 57.8% to 36.7%. This is the robustness–accuracy trade-off.
- Robustness transfers to ℓ_2 and ℓ_1 even though training used only ℓ_∞ (ℓ_2: 28.6 vs. 8.8, ℓ_1: 20.5 vs. 1.6).
- The adversarially trained model is not better at every ϵ: at 1/255 the standard model scores higher (37.5 vs. 34.6) because of its higher clean accuracy.
- The absolute accuracies are low (5 epochs, 10k images, small network, the adversarial training loss is still about 2.0).

(b) PGD vs. FGSM and different ϵ

FGSM is one attack step with no random start (`steps=1, rand_init=False`). PGD is 50 steps with 2 restarts. Accuracy under the ℓ_∞ attack:

ϵ	1/255	2/255	4/255	8/255	16/255
Standard, FGSM	39.4	26.7	12.6	2.6	0.5
Standard, PGD	37.5	21.9	4.8	0.2	0.0
ℓ_∞-AT, FGSM	34.6	32.8	28.9	23.9	16.4
ℓ_∞-AT, PGD	34.6	32.8	29.1	23.1	13.2

Accuracy under the ℓ_2 attack and under the ℓ_1 attack:

	ℓ_2, ϵ				ℓ_1, ϵ		
	0.25	0.5	1.0	2.0	6	12	24
Standard, FGSM	30.3	16.0	4.8	1.0	33.8	32.6	33.5
Standard, PGD	26.4	8.8	0.5	0.0	11.3	1.5	0.2
ℓ_∞-AT, FGSM	32.9	29.0	23.7	17.0	28.5	26.4	26.2
ℓ_∞-AT, PGD	32.9	28.6	22.5	12.1	26.6	20.4	11.9

1. PGD on the adversarially trained model vs. the standard model. In the ℓ_∞ table, the PGD accuracy of the adversarially trained model stays well above that of the standard model once ϵ is 2/255 or larger (32.8% vs. 21.9% at 2/255, 29.1% vs. 4.8% at 4/255, 23.1% vs. 0.2% at 8/255, 13.2% vs. 0.0% at 16/255). The same holds for ℓ_2 and ℓ_1. At 1/255 the standard model is better (37.5% vs. 34.6%), because of its higher clean accuracy (57.8% vs. 36.7%). So the adversarially trained model can still be attacked, but it is much harder to attack.

2. FGSM against both models. PGD is stronger than FGSM because it takes 50 steps (with 2 restarts) inside the same ϵ-ball and can reach a worse point than the single FGSM step. The adversarially trained model was trained against 10-step PGD, so a single step already finds almost as bad a point at small ϵ, and the gap appears only when ϵ grows.

- ℓ_∞, standard model: at 8/255, FGSM alone already brings the accuracy down to 2.6%, and PGD furthers it to 0.2%.
- ℓ_∞, adversarially trained model: at 8/255, FGSM (23.9%) is almost as strong as PGD (23.1%).
- ℓ_∞, adversarially trained model: the gap between FGSM and PGD grows with ϵ. It is zero at 1/255 and 2/255, and at 16/255 FGSM gives 16.4% and PGD 13.2%.

- ℓ_2, adversarially trained model: PGD and FGSM give similar accuracy at small ϵ (32.9% vs. 32.9% at 0.25), and the gap grows with ϵ (23.7% vs. 22.5% at 1.0, 17.0% vs. 12.1% at 2.0).
- ℓ_1: FGSM changes the accuracy of the standard model very little (about 33% at all three ϵ), while PGD brings it down to 0.2% at $\epsilon = 24$. On the adversarially trained model, FGSM gives 26.2% and PGD 11.9% at $\epsilon = 24$.

3. Different ϵ against the adversarially trained model (trained at $8/255$).

- The accuracy falls gradually as ϵ grows: 34.6%, 32.8%, 29.1%, 23.1% and 13.2% at 1/255, 2/255, 4/255, 8/255 and 16/255 (PGD), compared to 36.7% clean, with no sudden drop. The robustness is tied to the trained budget and degrades gradually beyond it.
- For comparison, the standard model falls faster: 37.5%, 21.9%, 4.8%, 0.2% and 0.0% at the same ϵ.
- ℓ_2 and ℓ_1 show the same gradual decline with ϵ under PGD (tables above).

P.T.O.

Problem 2. (SynCode)

The code is in a self-contained Colab notebook **here** and also on **GitHub**.

Approach. I use Microsoft's Phi-2 inside SynCode in `grammar_mask` mode. According to the question, the query has exactly two floating point operands and a single operator, which is one of the four operators $+$, $-$, $*$, $/$. So the grammar allows exactly two floats, each in parentheses, with one of these four operators between them, a space at the start and a newline at the end:

```
start: SP "(" NUM ")" OP "(" NUM ")" NL

NUM: /-?(0|[1-9][0-9]{0,8})\.[0-9]{1,8}/
OP: "+" | "-" | "*" | "/"
SP: " "
NL: "\n"
```

The `NUM` terminal is defined by the regular expression `-?(0|[1-9][0-9]{0,8})\.[0-9]{1,8}`:

- Sign, `-?`: an optional minus sign, so that negative operands are allowed.
- Integer part, `(0|[1-9][0-9]{0,8})`: either a single `0`, or a nonzero digit followed by up to eight more digits. This rules out leading zeros (such as `007.5`) and bounds the length of the integer part.
- Decimal point, `\.`: a literal dot, required in every operand.
- Fractional part, `[0-9]{1,8}`: one to eight digits. Requiring at least one digit forces the model to write numbers like `327.` in the form `327.0`.

The bounds on the number of digits also prevent unbounded repetition of digits.

The newline at the end lets the model "express" the end of the answer. Following the few-shot examples, the model tends to write a newline after the expression, before it stops. The grammar does not allow any other whitespace or a period, so the newline is the one legal way for the model to finish, and without it the model would be forced into some other token instead of ending. It is ignored afterwards by the `\s*` in the regular expression that parses the output.

Each operand is written in parentheses, so that the minus sign of a negative operand is not confused with the operator. `SP` is a single space at the start of the answer, to match the few-shot examples, where the answer follows `A:` after a space.

The prompt has few-shot examples that show only the extraction (the question and the expression). They are placed before the user query and reused for all queries:

```
Extract the two numbers and the operator as floats.
Q: What is 5. plus 2.5?
A: (5.0)+(2.5)
Q: What is 10.0 divided by 4.?
A: (10.0)/(4.0)
Q: What is -4.5 multiplied by 2.0?
A: (-4.5)*(2.0)
Q: What is -7.0 minus -3.25?
A: (-7.0)-(-3.25)
```

`run_syncode(textual_query)` appends the query to the few-shot prompt, lets SynCode generate the expression, parses it with a regular expression into the two operands and the operator, and computes the answer with Python:

```
PATTERN = re.compile(r"\s*(\(-?\d+\.\d+\))([+\-*/])(\(-?\d+\.\d+\))\s*")

def to_float(s):
    return float(s.strip("()"))

def run_syncode(textual_query):
    prompt = f"{FEWSHOT}Q: {textual_query}\nA:"
    raw = syn_llm.infer(prompt)[0]              # e.g. " (327.0)*(11.0)\n"
    m = PATTERN.fullmatch(raw)
    if m is None:
        raise ValueError(f"Unexpected SynCode output: {raw!r}")
    a, op, b = to_float(m.group(1)), m.group(2), to_float(m.group(3))
    return ([a, b], op, float(OPS[op](a, b)))   # Python does the math
```

It returns the tuple (ordered list of operands, operator, final answer).

Results. On the three examples of the problem and seven more queries (including negative operands and differently worded queries):

Query	Operands	Operator	Final answer	Expected
What is 327. multiplied by 11.0?	327.0, 11.0	*	3597.0	3597.0
What is 45.1 plus 23.54?	45.1, 23.54	+	68.64	68.64
What is 120.4 divided by 4.0?	120.4, 4.0	/	30.1	30.1
What is 90.5 minus 12.25?	90.5, 12.25	-	78.25	78.25
Multiply 3.2 by 4.0.	3.2, 4.0	*	12.8	12.8
What is -8.5 minus 2.0?	-8.5, 2.0	-	-10.5	-10.5
What is -2.5 multiplied by -4.0?	-2.5, -4.0	*	10.0	10.0
What is 6.0 minus -1.5?	6.0, -1.5	-	7.5	7.5
What is 99. divided by 4.?	99.0, 4.0	/	24.75	24.75
Subtract 3.5 from 10.0	10.0, 3.5	-	6.5	6.5

P.T.O.

Problem 3. (Multi-Norm Adversarial Training)

The code is in a Colab notebook **here** and also on **GitHub**.

The base model is the one given in the starter code: a PreActResNet-18 trained with ℓ_∞ PGD adversarial training for 30 epochs on all 50,000 CIFAR-10 training images. It is therefore already adversarially trained against ℓ_∞ perturbations. Each model below is fine-tuned from it for 10 epochs on a subset of 10,000 training images, with the default settings of `adv_train`. Accuracies (in %) are measured on the first 1,000 CIFAR-10 test images with the default evaluation (50 steps, 2 restarts), unless stated otherwise.

(a) Single-norm adversarial training

`single_norm_loss` attacks the batch with one norm and returns the cross-entropy loss of the model on the adversarial images:

```
def single_norm_loss(model, x, y, norm):
    model.eval()
    x_adv = attack(model, x, y, norm)
    model.train()
    return F.cross_entropy(model(x_adv), y)
```

Model	Clean	ℓ_∞	ℓ_2	ℓ_1	Union
Base	78.7	48.4	59.8	22.4	22.3
ℓ_∞-AT	78.3	44.8	57.0	20.2	20.0
ℓ_2-AT	82.3	32.2	60.2	28.2	24.3
ℓ_1-AT	84.3	0.0	0.1	26.8	0.0

(i) Weakest norm.

- Base and ℓ_∞-AT: ℓ_1 (22.4 and 20.2).
- ℓ_2-AT: ℓ_1 (28.2), then ℓ_∞ (32.2).
- ℓ_1-AT: ℓ_∞ and ℓ_2 (0.0 and 0.1).

The union accuracy is the lowest in every row, since an image counts for the union only if it is robust against all three norms.

(ii) Does robustness transfer? Only partly, and not evenly. The base model is already ℓ_∞ adversarially trained, so it serves as the reference for ℓ_∞ training.

- ℓ_∞ training transfers to ℓ_2 but hardly to ℓ_1: the base model has ℓ_2 accuracy 59.8 and ℓ_1 accuracy 22.4, and fine-tuning with ℓ_∞ attacks keeps this pattern (ℓ_2 57.0, ℓ_1 20.2).
- ℓ_2 training transfers partly to ℓ_1 (ℓ_1 accuracy rises from 22.4 to 28.2), but it reduces the ℓ_∞ accuracy from 48.4 to 32.2.
- ℓ_1 training does not transfer to the other norms: the ℓ_∞ and ℓ_2 accuracies fall from 48.4 and 59.8 to 0.0 and 0.1. The clean accuracy increases (84.3), which suggests that the model gave up its ℓ_∞ and ℓ_2 robustness.
- Fine-tuning on the 10k subset also reduces the ℓ_∞ accuracy of the base model (48.4 $\rightarrow$ 44.8 for ℓ_∞-AT).

(b) Multi-norm adversarial training (RAMP)

As specified in the question, I use $q = \infty$ and $r = 1$, with $\lambda = 0.5$ and $\beta = 0.5$. `ramp_loss` computes the mean of $\max(\mathcal{L}_{CE}(x^q), \mathcal{L}_{CE}(x^r))$ plus $\lambda \mathcal{L}_{KL}$, where $\mathcal{L}_{KL} = KL(p_q \,\|\, p_r)$ is computed only on the images that are classified correctly under the ℓ_∞ attack. `gradient_projection` returns $\cos(g_n, g_a)\, g_n$ if $\cos(g_n, g_a) > 0$ and 0 otherwise, for each parameter tensor.

```
def ramp_loss(model, x, y, lam=0.5):
    model.eval()
    x_q, x_r = attack(model, x, y, "linf"), attack(model, x, y, "l1")
    model.train()
    logits_q, logits_r = model(x_q), model(x_r)
    ce_q = F.cross_entropy(logits_q, y, reduction="none")
    ce_r = F.cross_entropy(logits_r, y, reduction="none")
    loss_adv = torch.maximum(ce_q, ce_r).mean()
    kl = F.kl_div(F.log_softmax(logits_r, 1), F.softmax(logits_q, 1), reduction="
    none").sum(1)
    ok = (logits_q.argmax(1) == y).float()           # correct under the linf
    attack
    loss_kl = (kl * ok).sum() / ok.sum().clamp_min(1.0)
    return loss_adv + lam * loss_kl

def gradient_projection(g_n, g_a):
    cos = (g_n * g_a).sum() / (g_n.norm() * g_a.norm() + 1e-12)
    return cos * g_n if cos > 0 else torch.zeros_like(g_n)
```

Model	Clean	ℓ_∞	ℓ_2	ℓ_1	Union	s / epoch
Base	78.7	48.4	59.8	22.4	22.3	–
ℓ_∞-AT	78.3	44.8	57.0	20.2	20.0	41.1
ℓ_2-AT	82.3	32.2	60.2	28.2	24.3	41.0
ℓ_1-AT	84.3	0.0	0.1	26.8	0.0	41.9
RAMP (seed 0)	79.3	41.0	61.9	44.6	38.8	88.7

(i) The highest union accuracy is RAMP (38.8), far above the best single-norm model (ℓ_2-AT, 24.3). The highest clean accuracy is ℓ_1-AT (84.3), then ℓ_2-AT (82.3). RAMP (79.3) is about the same as the base (78.7). The union gain of RAMP comes mainly from ℓ_1 ($22.4 \to 44.6$), at the cost of ℓ_∞ accuracy ($48.4 \to 41.0$).

(ii) RAMP takes about $2.2\times$ the time of single-norm training (88.7 s vs. 41.1 s per epoch). Each RAMP step attacks the batch twice (ℓ_∞ and ℓ_1) instead of once, and also computes an extra forward and backward pass on the clean batch for the gradient projection.

(c) Own method: weighted adversarial training over ℓ_∞ and ℓ_1

(i) Idea.

- An image counts as failed for the union accuracy if any one of the attacks causes a misclassification, so the union accuracy is limited by the weakest norm.
- An average of the losses of the norms does not show on which norm the model is currently weakest. The adversarial loss on each norm gives this information: a higher loss means that the model is currently weaker on that norm.
- Therefore, the per-norm losses are combined with weights that favour the norm on which the model is currently weaker, so that the updates put more emphasis on that norm.

In every batch I attack with ℓ_∞ and ℓ_1 and compute the cross-entropy losses $\mathcal{L}_\infty$ and $\mathcal{L}_1$. For each norm k, $\bar{\mathcal{L}}_k$ is the average of its loss over the last 20 batches (a moving average, so that the weights are not affected by the noise of a single batch). Then

$$w = \mathrm{softmax}\big(\tau\,[\bar{\mathcal{L}}_\infty,\ \bar{\mathcal{L}}_1]\big), \qquad \mathcal{L}_{\mathrm{prop}} = w_\infty\,\mathcal{L}_\infty + w_1\,\mathcal{L}_1,$$

with $\tau = 2$. The weights add up to 1 and are constants in the backward pass. $\tau = 0$ gives the plain average (weights 0.5 and 0.5), and a larger τ puts more weight on the norm with the higher loss.

I attack only with ℓ_∞ and ℓ_1, which are the two most different regions (a small change on every pixel vs. a large change on a few pixels). ℓ_2 lies between them, so it is expected to be covered as well. This also saves one attack per step.

(ii) Implementation. The loss is `my_loss` in the notebook, trained with `adv_train(load_base_model(), my_loss)` with the default settings, using $\tau = 2$ and a window of 20 batches:

```
def my_loss(model, x, y):
    norms = MY_CFG["norms"]                          # ("linf", "l1")
    model.eval()
    x_adv = {n: attack(model, x, y, n) for n in norms}
    model.train()
    losses = [F.cross_entropy(model(x_adv[n]), y) for n in norms]
    avg = []                                         # moving average over the
   last 20 batches
    for n, l in zip(norms, losses):
        h = MY_STATE["hist"].setdefault(n, deque(maxlen=MY_CFG["window"]))
        h.append(l.item())
        avg.append(sum(h) / len(h))
    w = torch.softmax(MY_CFG["tau"] * torch.tensor(avg, device=x.device), dim=0)
    return sum(w[k] * losses[k] for k in range(len(norms)))
```

(iii) Results. Mean of seeds 0 and 1 for the proposed method and for RAMP:

Model	Clean	ℓ_∞	ℓ_2	ℓ_1	Union	s / epoch
Base	78.7	48.4	59.8	22.4	22.3	–
RAMP	79.2	41.8	62.2	45.7	39.9	88.7
Proposed	77.8	41.5	62.1	48.3	40.2	82.7

Union per seed: proposed method 39.6 / 40.9, RAMP 38.8 / 41.0.

Over the runs, the proposed method (82.7 s per epoch) and RAMP (88.7 s per epoch) took practically the same time, so I did not train RAMP for more epochs.

(iv) Ablation (seed 0). I removed the adaptive weighting ($\tau = 0$, a plain average of the ℓ_∞ and ℓ_1 losses), and separately added an ℓ_2 attack.

Variant	Clean	ℓ_∞	ℓ_2	ℓ_1	Union	s / epoch
Proposed ($\tau = 2$)	77.8	41.0	62.0	47.4	39.6	82.7
$\tau = 0$ (uniform average)	78.1	40.7	61.7	48.4	39.6	82.6
+ ℓ_2 attack	78.9	40.7	62.0	47.3	39.2	123.7

(v) Analysis.

- **Versus the base model:** union improves from 22.3 to 40.2, mostly through ℓ_1 (22.4 → 48.3). ℓ_∞ drops (48.4 → 41.5), clean accuracy slightly (78.7 → 77.8), and ℓ_2 improves (59.8 → 62.1) without being trained on.

- **Versus RAMP:** union is comparable (40.2 vs. 39.9), ahead in seed 0 and 0.1 behind in seed 1, and the gap is within the test-set noise (about $\pm 1.5\%$). The method reaches the same level with a simpler update (no KL term, no gradient projection) and about 7% less time per epoch.
- **Weights:** in epoch 1, ℓ_1 gets more weight (0.53), matching the base model's weak ℓ_1. From epoch 2 on, ℓ_∞ gets more (0.54, then 0.57 from epoch 6), as ℓ_1 improves faster. Both seeds show this pattern.
- **Effect of weighting:** none on the union. Uniform weights give 39.6, the same as with weighting (39.6); the shift of accuracy toward ℓ_∞ (ℓ_∞ 40.7 $\rightarrow$ 41.0, ℓ_1 48.4 $\rightarrow$ 47.4) cancels in the union. Most of the gain comes from training on ℓ_1 and ℓ_∞ together.
- **Adding ℓ_2:** no gain (union 39.2 vs. 39.6) at 1.5$\times$ the time per epoch. Its weight stayed smallest (about 0.21), and ℓ_2 robustness is obtained without training on it.
- **Limitation:** ℓ_∞ is the weakest norm (41.5), and the union (40.2) sits just below it. A larger τ could push harder on it; this was not tested.

(vi) Evaluation check. The model of the proposed method with the higher union (seed 1, 40.9 vs. 39.6 for seed 0) was evaluated again with stronger attacks (`steps=200, restarts=5`, about 21 minutes, roughly 10$\times$ the default evaluation). It is the same model as in the table above, so the two rows are comparable.

Evaluation (seed=1)	Clean	ℓ_∞	ℓ_2	ℓ_1	Union
Default (50 steps, 2 restarts)	77.9	42.1	62.3	49.3	40.9
Strong (200 steps, 5 restarts)	77.9	41.9	62.3	46.7	40.0

The drops are small: 0.2 for ℓ_∞, 0.0 for ℓ_2, 2.6 for ℓ_1 and 0.9 for the union. If the default evaluation were overestimating robustness through gradient masking, the stronger attacks would take away much more. ℓ_1 is the norm that moves most.